Social & Media Platform APIs

Week 12 · APIs module · Textbook Chapter 12

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 2, 4 & 6, 2026

This week at a glance

Last week the data came from governments obligated to be open. This week it comes from private platforms that share it *at their discretion* — and can stop.

Monday | Concepts & notebook intro

- OAuth2, four platforms, and the governance spectrum from proprietary to open

Wednesday | Notebook lab

- Reddit, Spotify, Bluesky, and Mastodon — authenticate, retrieve, analyze

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 12

Companion notebook

`ch-12-social.ipynb`

Bring a laptop

Wednesday is hands-on — register your own apps and pull live data.

1. Monday · Concepts

Platforms, data, and power

Social platforms mediate an enormous share of public discourse. Their APIs are a **different animal** from last week's government APIs (Ch. 11): those exist because of transparency mandates; these exist at the discretion of a private company.

So access can change overnight — Reddit's 2023 pricing shift and Twitter's post-acquisition lockout are the cautionary cases.

We study four platforms spanning the spectrum from **proprietary** (Reddit, Spotify) to **open** (Bluesky, Mastodon) — the enclosure-vs-openness tension from the post-API age (Ch. 2).

Proprietary to open: who controls access.

Textbook Ch. 12, "Platforms, Data, and Power." Cross-refs: Government APIs (Ch. 11), the post-API age (Ch. 2).

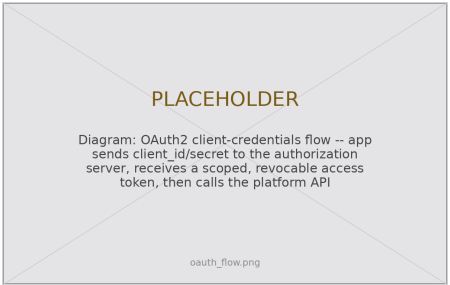
OAuth2: credentials for a scoped, revocable token

Almost every platform API authenticates with **OAuth2**. The idea: don't send your password with each request — exchange your app's credentials for a *scoped*, *revocable* token.

Two flows you should recognize:

- **Client credentials** — authenticates the *application*; grants read-only access to public data. Enough for research collection.
- **Authorization code** — additionally asks a *user* to grant your app permission to act for them. Every "Sign in with Google" dialog.

Store client IDs and secrets in environment variables, **never** in the notebook.



Bluesky app passwords and Mastodon tokens are the same delegated-credential idea, implemented more simply.

Textbook Ch. 12, "Authentication." Credential hygiene: *Missing Manual* Ch. 34.

Four platforms, four architectures

The differences are not cosmetic — they decide what data exists, on what terms, and how durable your access is:

Platform	Protocol	Auth	Rate limit	Data ownership
Reddit	Proprietary REST	OAuth2	100 QPM	Platform-controlled
Spotify	Proprietary REST	OAuth2 / client creds	∼180 RPM	Platform-controlled
Bluesky	AT Protocol *(open)*	App password	Generous, per-PDS	**User-portable**
Mastodon	ActivityPub *(open)*	OAuth2, per-instance	Varies	Instance-governed

Reddit and Spotify can *unilaterally* rewrite their terms because they own both the platform **and** the protocol. Bluesky and Mastodon cannot — their protocols are open standards anyone can implement. That is the architectural embodiment of **ownership** (Ch. 2).

Textbook Ch. 12, "Comparing Platform Architectures."

What the notebook covers

One authentication-to-DataFrame pattern, four times over:

- **Reddit (PRAW) —** subreddit metadata, submissions, comment trees, depth-vs-engagement
- **Spotify (spotipy) —** search, artist profiles, top-track popularity, user playlists
- **Bluesky (atproto) —** author feeds and the social graph (followers, following, mutuals)
- **Mastodon (Mastodon.py) —** public timelines, account search, and hashtags *across* instances

Every one: **authenticate → request → parse nested JSON → DataFrame.**

Connecting to Ch. 2 (Ethics): social data is people data

Every post, comment, and listening pattern was produced by a person who never imagined a research dataset. Check the ToS, consider IRB, collect the *minimum* you need, never touch private or DM content, and don't quote posts in ways a search engine can re-identify.

2. Wednesday · Notebook Lab

Reddit via PRAW — authenticate & explore

Client-only credentials put PRAW in read-only mode — exactly what research collection needs:

```
import praw

reddit = praw.Reddit(
    client_id="your_client_id",
    client_secret="your_client_secret",
    user_agent="WebDataScience/1.0 by u/your_username")
print(reddit.read_only) # True with app-only credentials

subreddit = reddit.subreddit("news")
print(f"Subscribers: {subreddit.subscribers:,}")
for rule in subreddit.rules: # PRAW returns Rule objects
    print("-", rule.short_name)
```

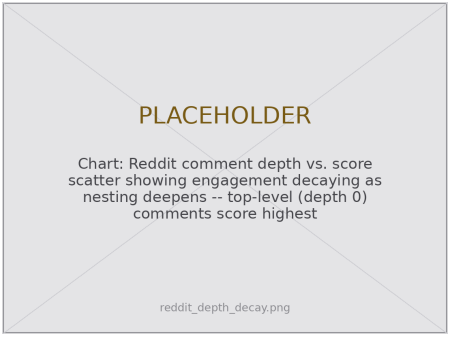
Textbook Ch. 12, “Reddit via PRAW.” Free tier: 100 queries/minute for non-commercial use.

Reddit — comment trees & the depth-decay pattern

```
sub = next(subreddit.hot(limit=1))
sub.comments.replace_more(limit=0) # flatten tree

rows = [{"score": c.score,
         "depth": c.depth,
         "words": len(c.body.split())}
        for c in sub.comments.list()]
df = pd.DataFrame(rows)

df.groupby("depth")["score"].mean()
# decays sharply as nesting deepens
```



Depth 0 comments score highest.

Not a measure of quality — an artifact of interface design, where nested replies are progressively collapsed and harder to reach.

Textbook Ch. 12, "Analyzing Comment Engagement." Score is skewed — prefer a Spearman rank correlation.

Spotify via spotipy — search & artist profile

The client-credentials flow reaches public catalog data — artists, albums, tracks, user playlists:

```
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials

auth = SpotifyClientCredentials(client_id="...", client_secret="...")
sp = spotipy.Spotify(auth_manager=auth)

results = sp.search(q="Queen", type="artist", limit=1)
artist = results["artists"]["items"][0]
top = sp.artist_top_tracks(artist["id"])["tracks"]
print(artist["name"], f'{artist["followers"]["total"]:,}',
      [t["popularity"] for t in top[:5]]) # popularity is 0-100
```

Textbook Ch. 12, “Spotify via spotipy.” `popularity` is engagement-weighted and recomputed on Spotify’s schedule.

The audio features story — enclosure in miniature

For a decade the API's signature research offering was **audio features** — danceability, energy, valence, tempo for every track. Hundreds of studies mapped genres and modeled hits with them.

November 2024: cut off for all newly created apps. No stated research alternative — just as generative AI made music data newly valuable.

The lasting lesson: those scores were never ground truth. They were Spotify's *proprietary model* of what music *is*. You always analyze the platform's model of the world as much as the world itself.

A shrinking API

New apps get `403 Forbidden` on audio features, recommendations, and related artists. It is not a bug in your code — the access no longer exists.

Textbook Ch. 12, "The Audio Features Story." Enclosure & the post-API age (Ch. 2).

Bluesky via the AT Protocol — open by design

```python
from atproto import Client

client = Client()
client.login("you.bsky.social",
             "your-app-password")

feed = client.app.bsky.feed.get_author_feed(
    {"actor": "bsky.app", "limit": 50})

posts = [{"text": i.post.record.text,
          "likes": i.post.like_count,
          "reply": i.post.record.reply is not None}
         for i in feed.feed]
df = pd.DataFrame(posts)
```

Social graph via `get_followers` / `get_follows`.

An **app password** is a revocable delegated credential — OAuth's principle, simpler. Identity and graph are portable across any AT-Protocol service.

Textbook Ch. 12, "Bluesky via AT Protocol." No single company controls access.

Mastodon via Mastodon.py — federated, per-instance

Each instance runs its own API — there is no single endpoint for "all of Mastodon." Statuses arrive as HTML, so strip tags before analysis:

```
from mastodon import Mastodon
from bs4 import BeautifulSoup

mastodon = Mastodon(access_token="user_cred.secret",
                    api_base_url="https://mastodon.social")

for toot in mastodon.timeline_public(limit=10):
    text = BeautifulSoup(toot["content"], "html.parser").get_text()
    print("@" + toot["account"]["username"], text[:80])

tagged = mastodon.timeline_hashtag("datascience", limit=20)
```

Comprehensive collection means engaging *many* instances — each with its own rules, rate limits, and moderation norms.

Lab: work these, then show-and-tell

Register your own apps, then work in pairs:

1. Reddit: compare two subreddits on one topic (r/science vs. r/askscience) — submissions, engagement, moderation
2. Spotify: analyze an artist's full discography — how has release cadence and back-catalog popularity shifted?
3. Open vs. closed: pull a profile, graph, and posts from *both* Bluesky and Reddit; compare ease and richness
4. Playlist fingerprints: two user-created playlists, overlapping metadata distributions
5. Bluesky: 50 posts via `get_author_feed` — summary stats & AT-Protocol limits
6. 5617: design a cross-platform measurement plan (proprietary vs. federated) — Bruns (2019), Davidson et al. (2023)

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

Platform goodwill is not infrastructure.

Twitter went from the most important data source in computational social science to inaccessible — in a year.

Build across open *and* closed, so no single shutdown ends your research.

3. Friday · Textbook Revisions

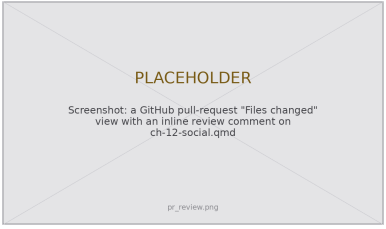
Improving Chapter 12 together

Today we make the book better. Each of you opens a **pull request** against `ch-12-social.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

This chapter drifts *fast* — endpoints and rate limits change — so "test it against the live API" is unusually valuable here.



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 12

High-value targets — pick one and scope it to a single PR:

- **Test the code against live APIs.** Register an app, run the notebook, and report what broke: a Spotify `403`, a changed `atproto` method, a new rate limit. Fix it and note the date.
- **Close a gap in "Common Issues to Debug."** Add a worked PRAW `OAuthException` fix, a Mastodon instance-block/rate-limit case, or the `403`-diagnosis flowchart.
- **Add a figure.** An OAuth token-flow diagram, the governance-spectrum figure, or a clean comment depth-vs-score chart.
- **Strengthen a thin exercise.** The Bluesky and playlist exercises need expected output, a rubric, or starter cells so they are self-checking.
- **Refresh the comparison table & "Further Reading."** Verify mid-2026 rate limits and endpoint availability; add a Fediverse-measurement paper tied to the **ownership** callout.

Targets drawn from the chapter's callouts, "Common Issues," thin exercises, and "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **AI & Language-Model APIs —** sending prompts to models as one more data pipeline.

Key takeaways

1. One pattern, every platform: **authenticate → request → parse nested JSON → DataFrame**.
2. OAuth2 trades credentials for a **scoped, revocable token**; client-credentials is enough for public research reads.
3. The real variable is **governance**, not code: who owns the data, on what terms, and what happens when the rules change.
4. Open protocols (AT, ActivityPub) make access an **architectural feature**; proprietary APIs make it a revocable business decision.
5. Access is conditional, costly, and fragile — build across open **and** closed so no single closure ends your research.

Reading: Textbook Ch. 12. Related: Baumgartner et al. (2020); Diener et al. (2026); Bruns (2019).